



Coding Agents in Noteable

Staff and Student Guidance

Academic Year 2026–27

Noteable now give students access to Codex, Claude and OpenCode coding agents.

These tools are available from the **command line** in all Noteable notebook environments and can support AI-assisted coding, debugging, explanation, notebook development and exploratory programming.

Noteable provides a safe and managed space for students and staff to experiment with AI-assisted and agentic programming. Behind the scenes, Noteable sessions run in containerised notebook environments, providing a strongly sandboxed workspace for code development and testing.

Note that while our Jupyter and RStudio environments are updated outside of teaching time, coding agents will be updated more frequently.

We recognise that requirements around AI use vary by course, cohort and assessment context. Noteable therefore provides flexibility for courses that wish to make use of these features.

For lecturers that do not wish to make coding agents available for their course/s, these can be disabled at institutional or course level.

Please test that the relevant AI tools for your course are available, whether Terminal-based access to Codex works as intended, and confirm that the available options support your intended teaching or workflow use cases.

What is a coding agent?

A **coding agent** is an AI-powered tool designed to help with software development tasks.

A coding agent is different from a general chatbot or standard large language model interface and it can often work more directly with files, code and the command line.

A coding agent may be able to:

- read files in a workspace;
- suggest and make code changes;
- edit or create files;
- explain code
- help debug errors;

- generate tests;
- help with notebooks and scripts;
- support multi-step programming tasks.

A general-purpose LLM usually responds to prompts in a chat interface. A coding agent can take actions in the user's workspace.

Coding agents available in Noteable

The Noteable Summer 2026 update includes access to:

- **Codex**
- **Claude**
- **OpenCode**

These are available from the terminal/command line in Noteable environments.

Noteable provides access to the coding-agent tools. Users will need to authenticate with, or provide an API key for, the relevant AI provider. Access requirements, available models, usage limits and any associated costs vary between coding agents and providers.

Any charges incurred through an AI provider are payable directly to that provider and are not included in a Noteable subscription.

Before using a coding agent in teaching, staff should confirm the required account, authentication and API arrangements, and test the intended workflow in the relevant Noteable environment.

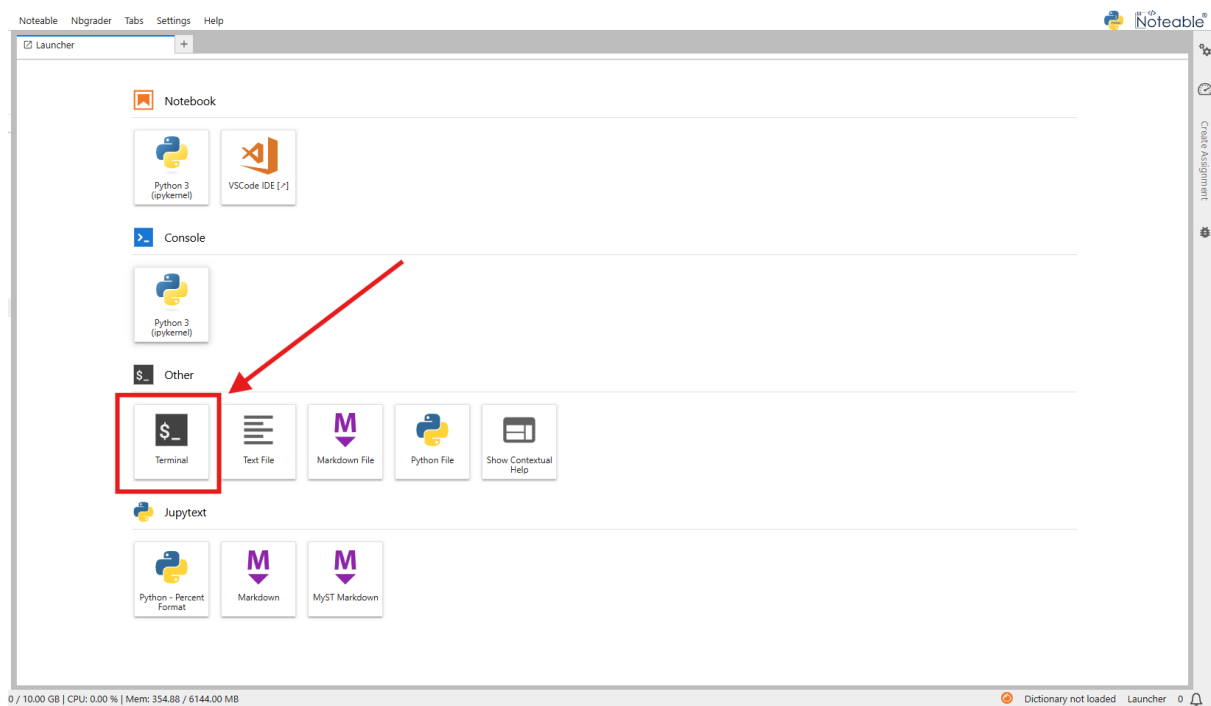
The video below goes over the set up process and getting started with coding agents using an example API key from EDINA's ELM[®] service.*

*See bottom of document for information about ELM.



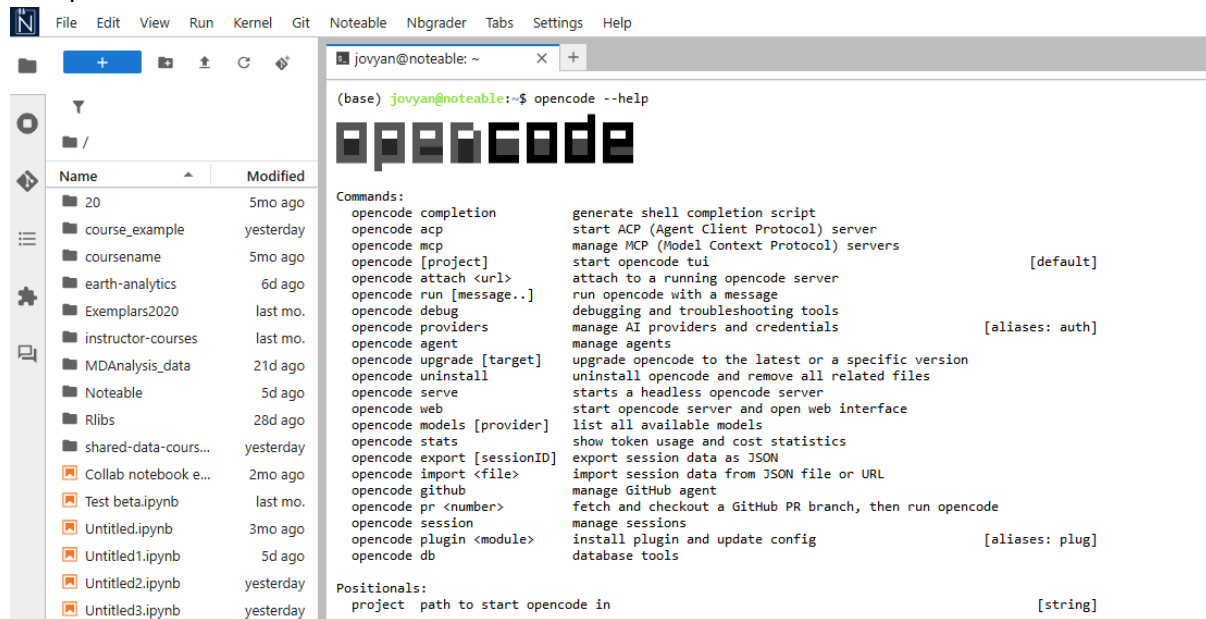
How to access coding agents in JupyterLab notebook servers:

1. Launch a Terminal



2. You can access coding agents directly through the Terminal, get started using <codingagent>

--help



The screenshot shows the Noteable JupyterLab interface. On the left is a file explorer with a list of files and folders. The main terminal window displays the output of the command `opencode --help`. The output includes the 'opencode' logo, a list of commands and their descriptions, and positional arguments.

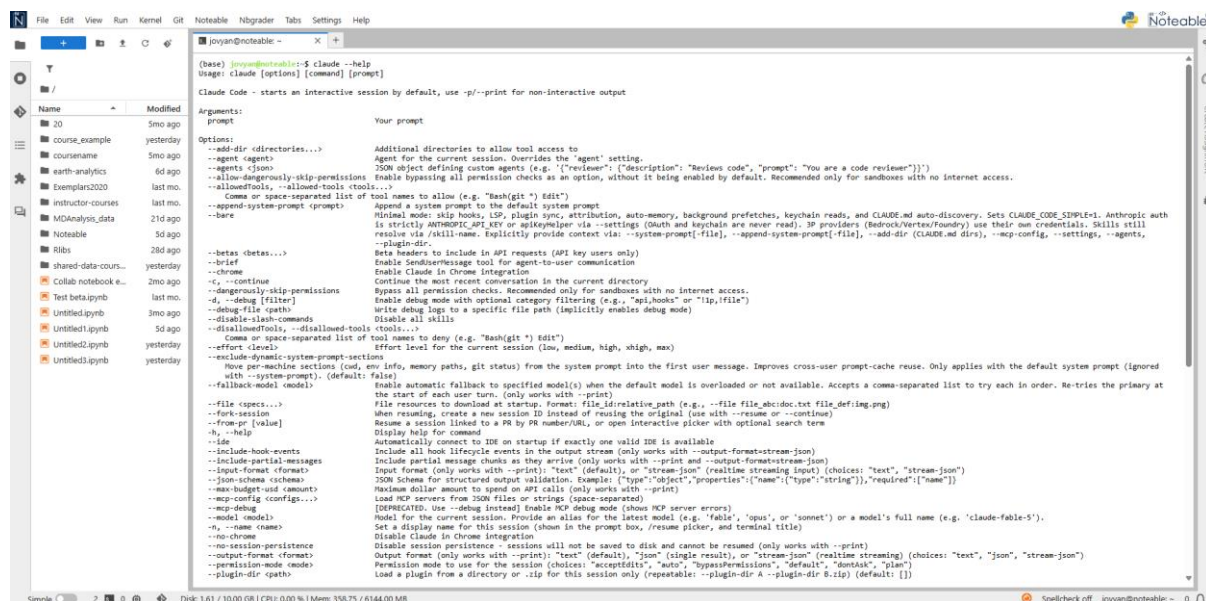
```
(base) jovyan@noteable: ~$ opencode --help

opencode

Commands:
  opencode completion      generate shell completion script
  opencode acp             start ACP (Agent Client Protocol) server
  opencode mcp             manage MCP (Model Context Protocol) servers
  opencode [project]      start opencode tui [default]
  opencode attach <url>   attach to a running opencode server
  opencode run [message..] run opencode with a message
  opencode debug           debugging and troubleshooting tools
  opencode providers       manage AI providers and credentials [aliases: auth]
  opencode agent          manage agents
  opencode upgrade [target] upgrade opencode to the latest or a specific version
  opencode uninstall      uninstall opencode and remove all related files
  opencode serve          starts a headless opencode server
  opencode web            start opencode server and open web interface
  opencode models [provider] list all available models
  opencode stats          show token usage and cost statistics
  opencode export [sessionID] export session data as JSON
  opencode import <file>  import session data from JSON file or URL
  opencode github         manage GitHub agent
  opencode pr <number>   fetch and checkout a GitHub PR branch, then run opencode
  opencode session        manage sessions
  opencode plugin <module> install plugin and update config [aliases: plug]
  opencode db             database tools

Positionals:
  project  path to start opencode in [string]
```

opencode in the Terminal in Noteable JupyterLab environments



The screenshot shows the Noteable JupyterLab interface. On the left is a file explorer. The main terminal window displays the output of the command `claude --help`. The output is a detailed list of arguments and options for the 'claude' command.

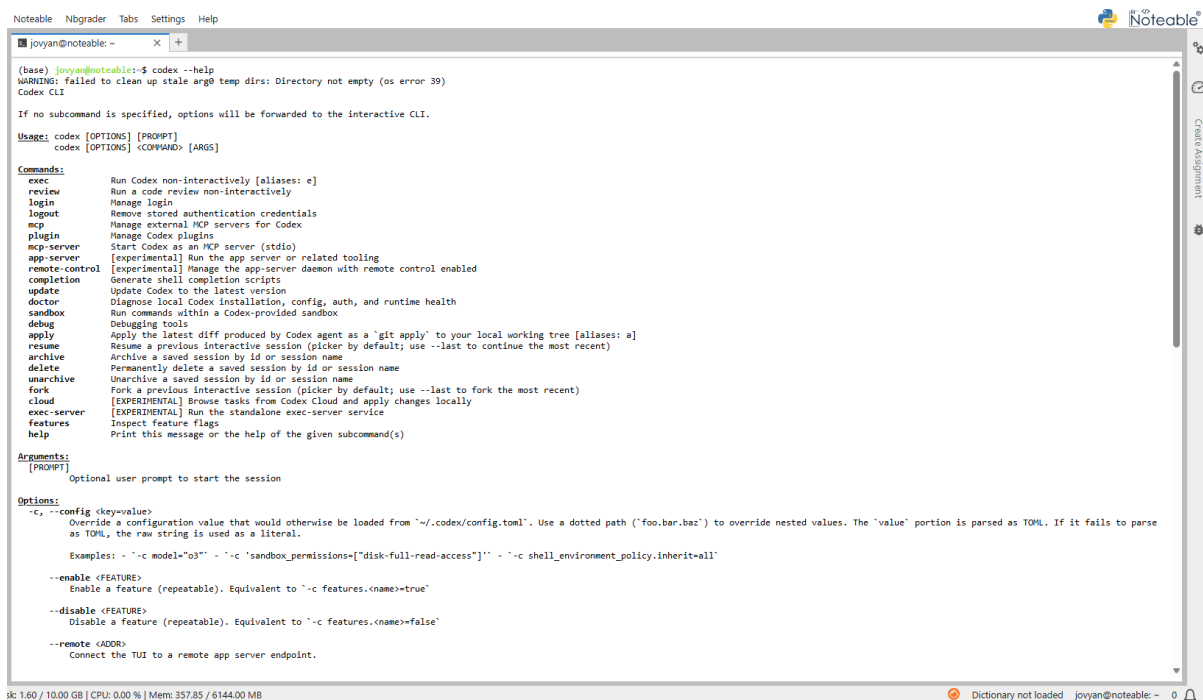
```
(base) [jovyan@noteable: ~]$ claude --help
Usage: claude [options] [command] [prompt]

Claude Code - starts an interactive session by default, use -p/-print for non-interactive output

Arguments:
  prompt          Your prompt

Options:
  --add-dir <directories...>  Additional directories to allow tool access to
  --agent <agents>           Agent for the current session. Overrides the 'agent' setting.
  --agents <json>           JSON object defining custom agents (e.g. '{"reviewer": {"description": "Review code", "prompt": "You are a code reviewer"}}')
  --allow-dangerously-skip-permissions  Enable bypassing all permission checks as an option, without it being enabled by default. Recommended only for sandboxes with no internet access.
  --allowed-tools, --allowed-tools <tools...>  Comma or space-separated list of tool names to allow (e.g. "bash(git *) Edit")
  --append-system-prompt <prompt>  Append a system prompt to the default system prompt
  --bare                Minimal mode: skip hooks, LSP, plugin sync, attribution, auto-memory, background prefetches, keychain reads, and CLAUDE.md auto-discovery. Sets CLAUDE_CODE_SIMPLE=1. Anthropic auth is strictly ANTHROPIC_API_KEY or spikyhelper via --settings (OAuth and keychain are never read). 3P providers (Bedrock/Vertex/Foundation) use their own credentials. Skills still resolve via //skill-name. Explicitly provide context via: --system-prompt[-file], --append-system-prompt[-file], --add-dir (CLAUDE.md dirs), --mcp-config, --settings, --agents, --plugin-dir.
  --beta <betas...>       Beta headers to include in API requests (API key users only)
  --chrome              Enable SendMessage tool for agent-to-user communication
  --continue            Continue the most recent conversation in the current directory
  --dangerously-skip-permissions  Bypass all permission checks. Recommended only for sandboxes with no internet access.
  -d, --debug <filter>   Enable debug mode with optional category filtering (e.g. "api,hooks" or "lsp,file")
  --debug-file <path>   Write debug log to a specific file path (implicitly enables debug mode)
  --disable-lash-commands  Disable all skills
  --disallowed-tools, --disallowed-tools <tools...>  Comma or space-separated list of tool names to deny (e.g. "bash(git *) Edit")
  --effort <level>       Effort level for the current session (low, medium, high, high, max)
  --exclude-dynamic-system-prompt-sections  Hide per-machine sections (cmd, env, info, memory paths, git status) from the system prompt into the first user message. Improves cross-user prompt-cache reuse. Only applies with the default system prompt (ignored with --system-prompt). (default: false)
  --fallback-model <model>  Enable automatic fallback to specified model(s) when the default model is overloaded or not available. Accepts a comma-separated list to try each in order. Re-tries the primary at the start of each user turn. (only works with --print)
  --file <specs...>       File resources to download at startup. Format: file_id:relative_path (e.g., --file file_id:doc.txt file_def:img.png)
  --fork-session         When resuming, create a new session ID instead of reusing the original (use with --resume or --continue)
  --from-pr <value>      Resume a session linked to a PR by PR number/URL, or open interactive picker with optional search term
  -h, --help             Display help for command
  --ide                 Automatically connect to IDE on startup if exactly one valid IDE is available
  --include-hook-events  Include all hook lifecycle events in the output stream (only works with --output-format=stream-json)
  --include-partial-messages  Include partial message chunks as they arrive (only works with --print and --output-format=stream-json)
  --input-format <format>  Input format (only works with --print): "text" (default), or "stream-json" (realtime streaming input) (choices: "text", "stream-json")
  --json-schema <schemas>  JSON Schema for structured output validation. Example: {"type": "object", "properties": {"name": {"type": "string"}, "required": ["name"]}}
  --max-budget-uid <amount>  Maximize dollar amount to spend on API calls (only works with --print)
  --mcp-debug            Load MCP servers from JSON files or strings (space-separated)
  --mcp-dir <dir>        [DEPRECATED: Use --debug instead] Enable MCP debug mode (shows MCP server errors)
  --model <model>        Model for the current session. Provide an alias for the latest model (e.g. 'fable', 'opus', or 'sonnet') or a model's full name (e.g. 'claude-fable-5').
  -n, --name <name>      Set a display name for this session (shown in the prompt box, /resume picker, and terminal title)
  --no-chrome            Disable Claude in Chrome integration
  --no-session-persistence  Disable session persistence - sessions will not be saved to disk and cannot be resumed (only works with --print)
  --output-format <format>  Output format (only works with --print): "text" (default), "json" (single result), or "stream-json" (realtime streaming) (choices: "text", "json", "stream-json")
  --permission-mode <mode>  Permission mode to use for the session (choices: "acceptAll", "auto", "bypassPermissions", "default", "dontAsk", "plan")
  --plugin-dir <paths>   Load a plugin from a directory or .zip for this session only (repeatable: --plugin-dir A --plugin-dir B.zip) (default: [])
```

Claude --help in the Terminal in JupyterLab environments



```
(base) joyan@noteable:~$ codex --help
WARNING: failed to clean up stale arg0 temp dirs: Directory not empty (os error 39)
Codex CLI

If no subcommand is specified, options will be forwarded to the interactive CLI.

Usage: codex [OPTIONS] [PROMPT]
       codex [OPTIONS] <COMMAND> [ARGS]

Commands:
  exec          Run Codex non-interactively [aliases: e]
  review       Run a code review non-interactively
  login        Manage login
  logout       Remove stored authentication credentials
  mcp          Manage external MCP servers for Codex
  plugin       Manage Codex plugins
  mcp-server   Start Codex as an MCP server (stdio)
  app-server   [experimental] Run the app server or related tooling
  remote-control [experimental] Manage the app-server daemon with remote control enabled
  completion   Generate shell completion scripts
  update       Update Codex to the latest version
  doctor       Diagnose local Codex installation, config, auth, and runtime health
  sandbox     Run commands within a Codex-provided sandbox
  debug        Debugging tools
  apply        Apply the latest diff produced by Codex agent as a 'git apply' to your local working tree [aliases: a]
  resume      Resume a previous interactive session (picker by default; use --last to continue the most recent)
  archive     Archive a saved session by id or session name
  delete      Permanently delete a saved session by id or session name
  unarchive   Unarchive a saved session by id or session name
  fork       Fork a previous interactive session (picker by default; use --last to fork the most recent)
  cloud      [EXPERIMENTAL] Browse tasks from Codex Cloud and apply changes locally
  exec-server [EXPERIMENTAL] Run the standalone exec-server service
  features    Inspect feature flags
  help       Print this message or the help of the given subcommand(s)

Arguments:
  [PROMPT]
    Optional user prompt to start the session

Options:
  -c, --config <key=value>
    Override a configuration value that would otherwise be loaded from '~/.codex/config.toml'. Use a dotted path ('foo.bar.baz') to override nested values. The 'value' portion is parsed as TOML. If it fails to parse as TOML, the raw string is used as a literal.

    Examples: - '-c model="os"' - '-c 'sandbox_permissions=["disk-full-read-access"]' - '-c shell_environment_policy.inherit=all'

  --enable <FEATURE>
    Enable a feature (repeatable). Equivalent to '-c features.<name>=true'

  --disable <FEATURE>
    Disable a feature (repeatable). Equivalent to '-c features.<name>=false'

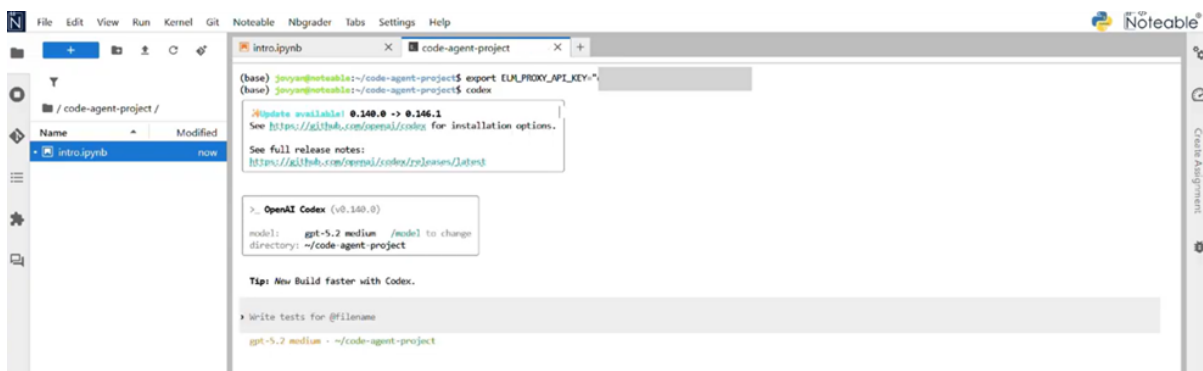
  --remote <ADDR>
    Connect the TUI to a remote app server endpoint.
```

Codex in the Terminal in Noteable JupyterLab environments

You can then write prompts and run relevant commands, for example:

`codex` starts the codex interactive session, while

`codex --help` displays available commands and usage options.



```
(base) joyan@noteable:~/code-agent-project$ export ELM_PROXY_API_KEY=""
(base) joyan@noteable:~/code-agent-project$ codex

Update available! 0.140.0 -> 0.146.1
See https://github.com/openai/codex for installation options.

See full release notes:
https://github.com/openai/codex/releases/latest

> OpenAI Codex (v0.140.0)

model: gpt-5.2 medium /model to change
directory: ~/code-agent-project

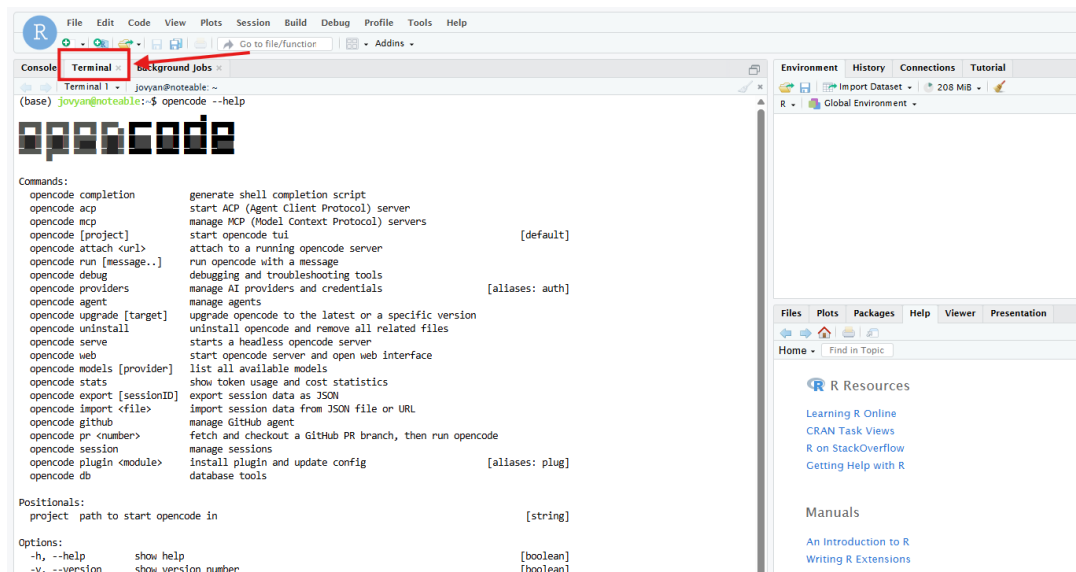
Tip: New Build faster with Codex.

Write tests for @filename

gpt-5.2 medium - ~/code-agent-project
```

In RStudio:

1. Launch your Noteable RStudio session.
2. Open the **Terminal** pane (found in the upper-left quadrant; if not visible, go to *Tools > Global Options > Terminal* or click the "Terminal" tab).
3. Run the agent commands directly (e.g., `opencode --help`, `claude --help`, or `codex --help`).
4. *Note:* These agents operate at the system level within the container, allowing them to assist with R script development, package management, and file operations directly from the RStudio terminal.



Coding agents may be updated more frequently than the main notebook environments. Notebook environments are usually updated outside teaching time, whereas coding agents may receive more frequent updates as provider tools and integrations change.

Why Noteable is a suitable place to use coding agents

Noteable is an excellent safe space for using coding agents because it provides a controlled teaching and learning environment.

Key features include:

- containerised notebook sessions;
- sandboxed user environments;
- consistent course environments;
- support for notebook-based teaching;
- options to enable or disable tools at institutional or course level.

This allows students to experiment with AI and agentic programming in a safer environment than working directly on a personal machine or unmanaged cloud system.

Testing checklist for staff

Before using coding agents in teaching, staff should test that:

- the required coding agents are available in the selected Noteable environment;
- terminal-based access works as intended;
- authentication works as expected;
- Codex, Claude and/or OpenCode support the intended workflow;
- students can access the relevant tools;
- the course position on AI use is clearly communicated;
- any required logging, declaration or acknowledgement process is in place.

ELM as an additional AI service

ELM® is EDINA's generative AI service: <https://elm.edina.ac.uk/>

For institutions interested in adding ELM® to their wider digital learning provision, it can support access to approved AI models and compatible coding-agent workflows through ELM® API keys.

To discuss ELM® for your institution, please contact EDINA.

Support and issue reporting

When reporting coding agent issues, please include:

- your university username;
- the notebook environment used;
- the course or test area used;
- the coding agent used;
- the command or workflow attempted;
- steps to reproduce the issue;
- screenshots or error messages.

Please report issues through the Noteable contact form:

<https://noteable.edina.ac.uk/contact-us/>